

DATA GOVERNANCE FOUNDATIONS

Data Standards, Policies & Documentation Practices

Data Dictionaries • Lineage • Metadata

Hands-on with Databricks & Unity Catalog

Created by Prachi Kabra

Instructor-led | Level: Intermediate | Duration: ~3 hours

Table of Contents

0. About This Module

This module equips data practitioners to design, apply, and enforce documentation standards across a modern lakehouse. Every concept is taught and practiced on the Databricks platform, using Unity Catalog as the governance and metadata backbone. By the end, learners will treat documentation not as an afterthought but as a first-class engineering deliverable.

Learning Objectives

- Explain what data standards, policies, and governance mean, and why they are business-critical.
- Define and apply naming conventions and data classification schemes.
- Build a living data dictionary in Unity Catalog using comments and tags.
- Read, interpret, and act on automated data lineage in Databricks.
- Distinguish technical, business, and operational metadata and manage each in the lakehouse.
- Query the Unity Catalog `information_schema` and system tables to audit documentation coverage.
- Use AI-assisted documentation (Databricks Assistant / AI-generated comments) responsibly.

Prerequisites

- Working knowledge of SQL and basic Python.
- A Databricks workspace with Unity Catalog enabled (Free Edition / Community or a Premium+ workspace).
- Ability to create a catalog, schema, and table (or CATALOG/SCHEMA permissions granted by an admin).

Environment note

All labs assume Unity Catalog. Lineage, tags, and system tables are Unity Catalog features and are not available on the legacy `hive_metastore`. If a command errors, confirm your notebook is attached to a UC-enabled cluster or SQL warehouse and that you are using a three-level name: `catalog.schema.object`.

1. Data Standards & Policies

1.1 What are data standards?

Data standards are the agreed-upon rules that govern how data is named, formatted, classified, described, and handled across an organization. They turn "everyone does it their own way" into a shared, predictable language. Policies are the enforceable commitments built on top of those standards — who may access what, how long data is retained, how quality is measured, and what happens when a rule is broken.

Standard = the rule ("all monetary columns are stored in minor units as BIGINT"). **Policy** = the enforced commitment with consequences ("PII columns must be tagged and masked; violations block deployment").

Why they matter

- Trust — consumers can rely on data meaning the same thing everywhere.
- Interoperability — teams and tools integrate without constant translation.
- Compliance — regulations (GDPR, HIPAA, RBI, SOX) demand demonstrable control over sensitive data.
- Efficiency — less time spent deciphering columns, more time delivering insight.
- Onboarding — new engineers become productive in days, not months.

1.2 Naming conventions

A naming convention is the single highest-leverage standard you can adopt: it is cheap to define and pays off on every object forever. Pick one style and apply it relentlessly.

Object	Convention	Example (Databricks)
Catalog	lowercase, environment or domain	prod, sales_analytics
Schema	lowercase snake_case, layer or subject	bronze, silver, gold, finance
Table	snake_case, singular subject + grain	fct_order_line, dim_customer
Column	snake_case, no reserved words	order_id, customer_email, amount_cents
Booleans	is_ / has_ prefix	is_active, has_consent
Dates	_date / _ts (timestamp) suffix	order_date, created_ts
Keys	_id suffix; surrogate _sk	customer_id, customer_sk

Medallion layering is the de-facto Databricks convention: **bronze** (raw, as-ingested), **silver** (cleaned, conformed), **gold** (business-ready, aggregated). Encode the layer in the schema name so lineage and purpose are obvious at a glance.

Rule of thumb

Names should be self-documenting but not a substitute for documentation. `amount_cents` tells you the unit; the column comment still explains the business meaning and source.

1.3 Data classification

Classification labels every data asset by its sensitivity so that access, masking, and retention policies can be applied consistently. A typical four-tier scheme:

Tier	Meaning	Examples	Typical control
Public	No harm if disclosed	Product catalog, press releases	Open read
Internal	Business-only	Sales dashboards, forecasts	Authenticated users
Confidential	Restricted, harmful if leaked	Contracts, pricing, salaries	Need-to-know + audit
Restricted / PII	Regulated personal / financial	PAN, Aadhaar, card, health	Masking + tagging + logging

In Databricks, classification is operationalised with Unity Catalog tags (key–value labels) plus governed tags for controlled vocabularies. You attach the class once and let policies reference the tag.

```
-- Classify a column as PII using a Unity Catalog tag
ALTER TABLE prod.silver.dim_customer
  ALTER COLUMN customer_email
    SET TAGS ('data_classification' = 'restricted', 'pii' = 'email');

-- Classify at the table level
ALTER TABLE prod.silver.dim_customer
  SET TAGS ('data_classification' = 'confidential', 'domain' = 'customer');
```

1.4 Governance policies in Unity Catalog

Policies are enforced through Unity Catalog’s privilege model (GRANT / REVOKE), row filters, and column masks. A few foundational patterns:

Access control (least privilege)

```
-- Grant read on a whole schema to an analyst group
GRANT USE CATALOG ON CATALOG prod TO `analysts`;
GRANT USE SCHEMA ON SCHEMA prod.gold TO `analysts`;
GRANT SELECT ON SCHEMA prod.gold TO `analysts`;

-- Revoke direct access to raw sensitive data
REVOKE SELECT ON TABLE prod.silver.dim_customer FROM `analysts`;
```

Column masking policy (protect PII on read)

```
-- 1) Define a reusable masking function
CREATE OR REPLACE FUNCTION prod.security.mask_email(email STRING)
RETURN CASE
  WHEN is_account_group_member('pii_readers') THEN email
  ELSE regexp_replace(email, '^(.)*.*$', '$1***$2')
END;

-- 2) Attach it to the column
ALTER TABLE prod.silver.dim_customer
  ALTER COLUMN customer_email
    SET MASK prod.security.mask_email;
```

Policy without documentation fails

A masking rule that no one can find is a landmine. Every policy should be documented: what it protects, who is exempt, and where it is applied. This is exactly why standards, policies, and documentation are taught together.

2. Documentation Practices

2.1 Why documentation is engineering, not paperwork

Undocumented data is a liability disguised as an asset. When the person who built a pipeline leaves, undocumented logic becomes tribal knowledge that erodes with every departure. Good documentation is version-controlled, lives close to the data, and is updated as part of the definition of done — never "later".

The three layers of documentation

Layer	Answers	Where it lives in Databricks
Object-level	What is this table/column?	UC COMMENT + tags
Pipeline-level	How is it produced?	DLT/Job docs, notebook markdown, Git README
Contract-level	What can I rely on?	Data contracts, expectations, SLA notes

2.2 Documentation as code

Treat documentation like source: store it in Git, review it in pull requests, and apply it through pipelines so it never drifts from reality. In Databricks you can embed COMMENT directly in DDL, keep it under version control, and re-apply on every deployment.

```
-- Documentation embedded in the table definition (idempotent)
CREATE TABLE IF NOT EXISTS prod.gold.fct_order_line (
  order_line_id BIGINT COMMENT 'Surrogate key, one row per line item',
  order_id      BIGINT COMMENT 'FK to fct_order; business order identifier',
  product_id    BIGINT COMMENT 'FK to dim_product',
  quantity      INT    COMMENT 'Units ordered; always > 0',
  amount_cents  BIGINT COMMENT 'Line revenue in minor units (INR paise)',
  order_ts      TIMESTAMP COMMENT 'Order creation time, UTC'
)
COMMENT 'Order line grain fact for revenue analytics. Source: silver.orders. Refresh: hourly.'
TBLPROPERTIES ('quality' = 'gold', 'owner' = 'revenue_analytics');
```

Notebook & pipeline documentation checklist

- Top-of-notebook markdown cell: purpose, inputs, outputs, owner, schedule.
- A README in the Git-backed repo describing the project and how to run it.
- Inline markdown before each major transformation explaining the "why".
- Data quality expectations (DLT EXPECT / constraints) documented alongside the rule.
- A changelog entry for every schema change.

The "future you" test

Write every comment for a competent colleague — or yourself in six months — who has zero context. If they would still have to ask "why?", the documentation is not done.

3. Data Dictionaries

3.1 What is a data dictionary?

A data dictionary is the authoritative catalog of your data assets — every table and column described in business terms, with its type, source, classification, owner, and rules. It is the single reference a new analyst opens to understand "what does amount_cents actually mean and can I trust it?"

Core components of a dictionary entry

Attribute	Description	Example
Name	Physical object/column name	dim_customer.customer_email
Business definition	Plain-language meaning	Primary contact email of the customer
Data type	Physical type	STRING
Nullability	Required or optional	Nullable
Classification	Sensitivity tier	Restricted / PII
Source	System of origin / lineage	CRM → bronze.customers
Owner	Accountable steward	customer_domain team
Valid values / rules	Domain & constraints	Valid email format; unique

3.2 Building a data dictionary in Unity Catalog

The most durable dictionary is the one that lives with the data. In Databricks, comments and tags on UC objects ARE your dictionary — and because they are queryable through information_schema, you can generate a full dictionary report on demand instead of maintaining a stale spreadsheet.

Step 1 — Describe objects with COMMENT

```
COMMENT ON TABLE prod.silver.dim_customer IS
  'One row per customer. Conformed from CRM + web signups. SCD Type 2.';

COMMENT ON COLUMN prod.silver.dim_customer.customer_id IS
  'Natural business key from the CRM system';

COMMENT ON COLUMN prod.silver.dim_customer.customer_email IS
  'Primary contact email. PII. Masked for non-privileged readers.';
```

Step 2 — Add machine-readable tags

```
ALTER TABLE prod.silver.dim_customer
  SET TAGS ('domain'='customer', 'steward'='customer_team', 'cert'='certified');

ALTER TABLE prod.silver.dim_customer
  ALTER COLUMN customer_email
  SET TAGS ('classification'='restricted', 'pii'='email');
```

Step 3 — Generate the dictionary from metadata

Because Unity Catalog exposes an ANSI information_schema, one query produces a live column dictionary:

```
SELECT
  c.table_catalog,
  c.table_schema,
  c.table_name,
```

```

    c.column_name,
    c.full_data_type          AS data_type,
    c.is_nullable,
    c.comment                 AS business_definition
FROM prod.information_schema.columns c
WHERE c.table_schema = 'silver'
ORDER BY c.table_name, c.ordinal_position;

```

Join in tags to enrich the dictionary with classification:

```

SELECT
    t.table_name,
    t.column_name,
    t.tag_name,
    t.tag_value
FROM prod.information_schema.column_tags t
WHERE t.schema_name = 'silver'
ORDER BY t.table_name, t.column_name;

```

Step 4 — Export with Python (optional)

```

# Databricks notebook (PySpark) – export a dictionary to a Delta table
df = spark.sql("""
    SELECT table_catalog, table_schema, table_name, column_name,
           full_data_type AS data_type, is_nullable,
           comment AS business_definition
    FROM prod.information_schema.columns
    WHERE table_schema IN ('silver','gold')
""")

(df.write
 .mode("overwrite")
 .saveAsTable("prod.governance.data_dictionary"))

# Flag undocumented columns – your documentation coverage KPI
missing = df.filter("business_definition IS NULL OR business_definition = ''")
print(f"Undocumented columns: {missing.count()}")
display(missing)

```

Catalog Explorer

Everything above is also editable in the Databricks Catalog Explorer UI — comments, tags, and AI-suggested descriptions. Teach both: SQL/Python for automation and repeatability, the UI for quick stewardship by non-engineers.

4. Data Lineage

4.1 What is data lineage?

Data lineage is the map of where data comes from, how it moves, and how it is transformed — from source system to final dashboard. It answers three questions that every data team hears weekly: "Where did this number come from?", "What breaks if I change this column?", and "Which report is affected by this bad upstream load?"

Granularity	What it shows	Use case
Table-level	Which tables feed which tables	Impact analysis, dependency mapping
Column-level	Which source columns build a target column	Root-cause, PII propagation tracing
Notebook/Job	Which code produced an asset	Ownership, reproducibility

4.2 Automatic lineage in Unity Catalog

This is a major advantage of Databricks: Unity Catalog captures lineage automatically for any workload that runs against UC — SQL, Python, Scala, notebooks, jobs, and DLT pipelines. You do not annotate it by hand; you read it. Lineage is captured down to the column level and visualized in Catalog Explorer.

Viewing lineage in the UI

1. Open Catalog Explorer and navigate to the table.
2. Select the Lineage tab to see upstream and downstream tables.
3. Click See lineage graph for the interactive diagram.
4. Click a column to trace column-level lineage to its sources.

Querying lineage programmatically (system tables)

Lineage is also exposed as queryable system tables, so you can audit and automate impact analysis:

```
-- Table-level lineage: what feeds prod.gold.fct_order_line?
SELECT source_table_full_name, target_table_full_name, event_time
FROM system.access.table_lineage
WHERE target_table_full_name = 'prod.gold.fct_order_line'
ORDER BY event_time DESC;

-- Column-level lineage: trace a single column's origins
SELECT source_table_full_name, source_column_name,
       target_table_full_name, target_column_name
FROM system.access.column_lineage
WHERE target_table_full_name = 'prod.gold.fct_order_line'
AND target_column_name = 'amount_cents';
```

Impact analysis in one query

Before altering or dropping a column, query downstream lineage. If a marketing dashboard depends on it, you will know before you break it — not after the escalation email.

Lineage + classification = PII propagation tracking

Combine lineage with your PII tags to answer a compliance-grade question: "everywhere a piece of personal data has flowed." This is where the modules connect — classification (Module 1) rides along the lineage graph (Module 4) to prove regulatory control.

5. Metadata Management

5.1 The three kinds of metadata

Metadata is "data about data". Managing it deliberately is what turns a data swamp into a governed lakehouse. Three categories matter:

Type	Answers	Databricks source
Technical	Structure: types, schema, size, format	information_schema, DESCRIBE, system tables
Business	Meaning: definitions, owners, domain, KPIs	Comments, tags, certifications
Operational	Behaviour: freshness, volume, quality, cost	system tables, DLT metrics, job runs

5.2 Technical metadata

```
-- Structural metadata for a table
DESCRIBE EXTENDED prod.gold.fct_order_line;

-- Programmatic view across the catalog
SELECT table_schema, table_name, table_type, created, last_altered
FROM prod.information_schema.tables
WHERE table_schema = 'gold'
ORDER BY last_altered DESC;
```

5.3 Operational metadata (freshness, volume, quality)

Unity Catalog system tables record how your data behaves over time — invaluable for SLAs and cost control.

```
-- When was each table last written to? (freshness)
SELECT table_name, MAX(event_time) AS last_write
FROM system.access.table_lineage
WHERE target_table_schema = 'gold'
GROUP BY table_name
ORDER BY last_write DESC;

-- Pipeline / job execution history for reliability metrics
SELECT * FROM system.lakeflow.job_run_timeline
ORDER BY period_start_time DESC
LIMIT 50;
```

5.4 AI-assisted metadata

Databricks can generate draft column and table comments using AI (visible as suggested descriptions in Catalog Explorer, and via the Databricks Assistant). This dramatically lowers the cost of documenting a large legacy estate.

Using AI comments responsibly

- Treat AI suggestions as a first draft, never as final truth.
- A steward must review and approve before a description becomes "certified".
- AI infers meaning from names and sample data — it cannot know your business rules, so verify units, grain, and edge cases.
- Never let AI generate classification tags for sensitive data unreviewed; a missed PII tag is a compliance failure.

5.5 Auditing documentation coverage

What gets measured gets documented. Turn coverage into a KPI and track it:

```
-- Documentation coverage %: columns with a non-empty comment
SELECT
  table_schema,
  COUNT(*)                                AS total_columns,
  COUNT(NULLIF(TRIM(comment), ''))        AS documented,
  ROUND(100.0 * COUNT(NULLIF(TRIM(comment), '')) / COUNT(*), 1) AS pct_documented
FROM prod.information_schema.columns
WHERE table_schema IN ('silver','gold')
GROUP BY table_schema
ORDER BY pct_documented;
```

Governance dashboard idea

Schedule the coverage query as a Databricks SQL dashboard. A visible "documentation coverage" gauge per domain creates gentle, continuous pressure to keep the dictionary complete.

6. Hands-On Lab

Complete this end-to-end lab in your Databricks workspace. It ties all five modules together on a single dataset. Allow ~45 minutes.

Lab objective

Create a small governed dataset, document it as a data dictionary, classify PII, inspect automatic lineage, and produce a documentation-coverage report.

Step 1 — Set up governed objects

```
CREATE CATALOG IF NOT EXISTS training;
CREATE SCHEMA IF NOT EXISTS training.bronze;
CREATE SCHEMA IF NOT EXISTS training.silver;

CREATE TABLE training.bronze.raw_customers (
  id INT, name STRING, email STRING, country STRING, signup_ts TIMESTAMP
);

INSERT INTO training.bronze.raw_customers VALUES
  (1, 'Asha Rao', 'asha@example.com', 'IN', current_timestamp()),
  (2, 'Liam Ford', 'liam@example.com', 'US', current_timestamp());
```

Step 2 — Build a documented silver table (dictionary)

```
CREATE TABLE training.silver.dim_customer
COMMENT 'Conformed customer dimension. Source: bronze.raw_customers.'
AS SELECT
  id          AS customer_id,
  name        AS customer_name,
  email       AS customer_email,
  country     AS country_code,
  signup_ts   AS signup_ts
FROM training.bronze.raw_customers;

COMMENT ON COLUMN training.silver.dim_customer.customer_id IS 'Business key';
COMMENT ON COLUMN training.silver.dim_customer.customer_email IS 'Contact email. PII.';
COMMENT ON COLUMN training.silver.dim_customer.country_code IS 'ISO 3166-1 alpha-2';
```

Step 3 — Classify PII with tags

```
ALTER TABLE training.silver.dim_customer
  ALTER COLUMN customer_email
  SET TAGS ('classification'='restricted', 'pii'='email');
```

Step 4 — Inspect lineage

Open Catalog Explorer → training.silver.dim_customer → Lineage tab. Confirm that bronze.raw_customers appears upstream and trace customer_email at the column level. (System-table lineage may take a few minutes to populate.)

Step 5 — Documentation coverage report

```
SELECT column_name, comment
```

```
FROM training.information_schema.columns
WHERE table_schema='silver' AND table_name='dim_customer';

-- Coverage %
SELECT ROUND(100.0*COUNT(NULLIF(TRIM(comment),''))/COUNT(*),1) AS pct_documented
FROM training.information_schema.columns
WHERE table_schema='silver' AND table_name='dim_customer';
```

Deliverable

Submit: (1) a screenshot of the lineage graph, (2) the output of the coverage query, and (3) a one-paragraph note on which column you would mask and why.

7. Knowledge Check

Use these to check understanding at the end of the session. Answers appear on the following page.

1. Explain the difference between a data standard and a data policy, with one example of each.
2. You must store customer PAN numbers. Which classification tier applies, and what two controls should accompany it?
3. Name the three medallion layers and describe the purpose of each.
4. Where does a data dictionary "live" in Databricks, and why is that better than a spreadsheet?
5. Which `information_schema` view lists column comments, and how would you find undocumented columns?
6. A teammate wants to drop `dim_customer.customer_email`. What must you check first, and using which capability?
7. Give one example each of technical, business, and operational metadata.
8. Why should AI-generated column comments be reviewed before being certified?

Answer Key

- 1.** A standard is the agreed rule (e.g. "money is stored in minor units as BIGINT"). A policy is the enforced commitment with consequences (e.g. "PII columns must be masked; violations block deployment").
- 2.** Restricted / PII. Controls: a Unity Catalog classification tag plus a column mask (and access limited to a privileged group with audit logging).
- 3.** Bronze = raw, as-ingested data. Silver = cleaned and conformed. Gold = business-ready, aggregated for consumption.
- 4.** It lives as comments and tags on Unity Catalog objects, queryable via `information_schema`. Unlike a spreadsheet it cannot drift from reality and is generated on demand.
- 5.** `information_schema.columns` exposes the comment field; filter where comment IS NULL OR `TRIM(comment)=''` to list undocumented columns.
- 6.** Check downstream lineage (Catalog Explorer Lineage tab or `system.access.column_lineage`) to see what depends on the column before dropping it.
- 7.** Technical: data type / schema. Business: the column definition and owner. Operational: last-write freshness or job run success rate.
- 8.** AI infers meaning from names and sample data and cannot know business rules; an unreviewed comment can be subtly wrong, and a missed PII tag is a compliance failure.

8. Best-Practice Checklist

A one-page reference learners can pin above their desk.

- Adopt one naming convention and apply it to every catalog, schema, table, and column.
- Use medallion layering (bronze / silver / gold) in schema names.
- Classify every dataset into a sensitivity tier and tag it in Unity Catalog.
- Write a COMMENT for every table and every column — no exceptions for gold.
- Store DDL and documentation in Git; apply it through deployment pipelines.
- Generate your data dictionary from information_schema, not a spreadsheet.
- Read lineage before altering or dropping any column (impact analysis).
- Protect PII with tags plus column masks, and trace it along lineage.
- Track documentation coverage % as a KPI on a scheduled dashboard.
- Treat AI-generated documentation as a draft requiring human certification.

Key takeaway

Standards, policies, and documentation are not three separate chores — they are one governance loop. In Databricks, Unity Catalog is where all three converge: it stores the rules, enforces the policies, and generates the documentation automatically. Teach them together and they reinforce each other.

— End of Module —

Created by Prachi Kabra • Data Governance Foundations